🚀 Complete JWT Flow in Microservices — From Login to Authorization (Deep Explanation)

When we design authentication for microservices, JWT becomes the backbone of secure and scalable communication.
Here's the full journey of a JWT — from login to protected microservice access — explained step-by-step.

◆ 1⃣ What Happens in the Background When User Sends Username & Password?

When a login request hits the **Auth Service**, a full security pipeline starts:

□ A. Authentication Filter Triggers

- The request reaches Spring Security's `UsernamePasswordAuthenticationFilter`
- Extracts username & password from the request body

□ B. Authentication Manager is Invoked

- `AuthenticationManager` sends credentials to the **Authentication Provider**

□ C. UserDetailsService Loads User

- Database lookup using `loadUserByUsername()`
- Retrieves username, hashed password, roles, active status, etc.

□ D. Password Verification

- Entered password is matched with DB password using `PasswordEncoder` (BCrypt recommended)
- If mismatch → Authentication exception
- If match → User is authenticated successfully

□ E. JWT Creation

Auth service now generates the JWT containing:

- `sub` (username)
- `roles` / authorities

- `iat` (issued time)
- `exp` (expiry time)
- Custom claims (userId, email, department etc.)

Finally, **JWT is digitally signed**:

- With **HMAC (HS256)** → secret key
- OR **RSA (RS256)** → private key

☞ Only the Auth Service has the private key → token cannot be forged.

◆ 2 How the Client Uses the JWT?

JWT is stored in:

- `localStorage`, or
- `sessionStorage`

Sent with every request:

```
Authorization: Bearer <TOKEN>
```

No session on the server → System remains **stateless**.

◆ 3 API Gateway — Validation + Routing Layer

Every request must pass through API Gateway before reaching any microservice.

Here's what happens internally:

🔧 A. API Gateway Extracts and Validates JWT

A custom filter (e.g., Spring Cloud Gateway GlobalFilter) performs:

✓ Signature validation

- Confirms token is signed by Auth Service
- If RSA → validates using **public key**

- If invalid signature → reject immediately

✓ Expiry validation

- Rejects expired tokens
- Ensures user logs in again or uses refresh token

✓ Claims validation

- Extracts username
- Extracts roles
- Validates issuer, audience, timestamps, etc.

🚦 B. Routing to Correct Microservice

After validation, Gateway applies routing rules defined in config:

```
/users/**       → User-Service
/orders/**      → Order-Service
/payments/**    → Payment-Service
```

Routing is controlled through:

- `application.yml` routes
- Predicates (path, method)
- Filters (JWT Filter goes here)

📨 C. Forwarding User Context

Gateway adds identity info into headers:

```
X-User-Id
X-User-Role
X-User-Email
```

So microservices don't need to parse JWT again.

◆ 4 Microservices — Authorization Layer

Inside each microservice:

□ A. Business Logic Starts After Gateway Validation

No JWT parsing required unless architecture demands re-validation.

□ B. Role-Based Access

Spring Security annotations enforce authorization:

```
@PreAuthorize("hasRole('ADMIN')")
```

□ C. Microservices Stay Stateless

They do not maintain sessions or store user login data.

◆ 5 Refresh Token Flow (Optional but Recommended)

To avoid frequent login:

- Access token → short-lived
- Refresh token → long-lived

Used to get a new access token without re-entering password.